

Contents

Manual Técnico: Funciones Agregadas en Bridge GeCode	2
Implementación Modular de Funciones Agregadas en C	2
1. Introducción	2
1.1 Objetivos	2
1.2 Arquitectura Modular	2
2. Diseño de la Solución	2
2.1 Principios de Diseño	2
2.2 Estructura de Archivos	3
3. Funciones Agregadas Implementadas	4
3.1 Funciones Numéricas	4
3.2 Funciones para Conjuntos	4
3.3 Funciones Lógicas	4
3.4 Funciones Enteras	4
4. Proceso de Compilación	5
4.1 Modificación del Makefile	5
4.2 Pasos de Compilación	5
4.3 Verificación	5
5. Uso de Funciones Agregadas	6
5.1 Desde Código C	6
5.2 Desde Expresiones JSON (Futuro)	6
6. Detalles de Implementación	6
6.1 SUM - Suma de Elementos	6
6.2 AVG - Promedio	7
6.3 MEDIAN - Mediana	7
6.4 VARIANCE - Varianza	7
6.5 ALL/ANY/NONE - Operadores Lógicos	8
7. Ventajas de la Arquitectura Modular	8
7.1 Separación de Responsabilidades	8
7.2 Beneficios	8
8. Integración con el Parser (Próximos Pasos)	9
8.1 Modificar bridge_gecode.y	9
8.2 Modificar bridge_gecode.l	9
8.3 Extender expr_eval.c	9
9. Ejemplos de Uso	10
9.1 Estadísticas Descriptivas	10
9.2 Análisis de Conjuntos	10
9.3 Validación Lógica	10
10. Testing y Validación	11
10.1 Test Unitario Básico	11
10.2 Compilar y Ejecutar Tests	11
11. Documentación de API	11
11.1 Convenciones	11
11.2 Complejidad Computacional	11
12. Conclusiones	12

12.1 Logros	12
12.2 Próximos Pasos	12
12.3 Recursos	12

Manual Técnico: Funciones Agregadas en Bridge GeCode

Implementación Modular de Funciones Agregadas en C

Fecha: 21 de febrero de 2026 **Autor:** Bridge GeCode Development Team
Versión: 1.0

1. Introducción

Este documento describe la implementación de **funciones agregadas** (aggregate functions) como módulo separado en el sistema Bridge GeCode Validator. Las funciones agregadas permiten realizar operaciones sobre colecciones de datos (arrays, conjuntos) como SUM, AVG, MIN, MAX, etc.

1.1 Objetivos

- Implementar funciones agregadas como **módulo independiente**
- Generar **código objeto** (.o) que se linke con el proyecto principal
- Permitir uso de funciones en **expresiones JSON**
- Mantener **separación de responsabilidades**
- Facilitar **extensibilidad** futura

1.2 Arquitectura Modular

```
Bridge GeCode
+-- bridge_gecode.y/l      (Parser principal)
+-- expr_eval.c            (Evaluador de expresiones)
+-- json_reader.c          (Lector JSON)
+-- aggregate_functions.c  (NUEVO: Funciones agregadas)
+-- aggregate_functions.h  (NUEVO: Interfaz pública)
```

2. Diseño de la Solución

2.1 Principios de Diseño

1. Separación de Módulos

- Las funciones agregadas están en archivos separados
- Interfaz clara (header .h)

- Implementación independiente (.c)
2. **Compilación Independiente**
 - Cada módulo se compila a código objeto (.o)
 - Linkeo en tiempo de compilación
 - Sin dependencias circulares
 3. **Reutilización**
 - Funciones pueden usarse desde cualquier parte del proyecto
 - API estable y documentada

2.2 Estructura de Archivos

aggregate_functions.h (Interfaz)

```
#ifndef AGGREGATE_FUNCTIONS_H
#define AGGREGATE_FUNCTIONS_H

#include <stddef.h>

/* Funciones numéricas */
double aggregate_sum(const double *array, size_t length);
double aggregate_avg(const double *array, size_t length);
double aggregate_min(const double *array, size_t length);
double aggregate_max(const double *array, size_t length);
// ... más funciones

#endif
```

Características: - Guardas de inclusión (#ifndef) - Tipos estándar (size_t) - const para arrays de solo lectura - Documentación en comentarios

aggregate_functions.c (Implementación)

```
#include "aggregate_functions.h"
#include <math.h>
#include <stdlib.h>

double aggregate_sum(const double *array, size_t length) {
    if (!array || length == 0) return 0.0;

    double sum = 0.0;
    for (size_t i = 0; i < length; i++) {
        sum += array[i];
    }
    return sum;
}

// ... implementaciones
```

Características: - Validación de parámetros - Manejo de casos límite - Código optimizado

3. Funciones Agregadas Implementadas

3.1 Funciones Numéricas

Función	Descripción	Entrada	Salida
aggregate_sum	Suma de elementos	double[]	double
aggregate_avg	Promedio aritmético	double[]	double
aggregate_min	Valor mínimo	double[]	double
aggregate_max	Valor máximo	double[]	double
aggregate_product	Producto	double[]	double
aggregate_median	Mediana	double[]	double
aggregate_variance	Varianza poblacional	double[]	double
aggregate_stdev	Desviación estándar	double[]	double

3.2 Funciones para Conjuntos

Función	Descripción	Entrada	Salida
aggregate_count	Cuenta elementos	char*[]	size_t
aggregate_count_if	Cuenta con condición	char*[]	size_t

3.3 Funciones Lógicas

Función	Descripción	Entrada	Salida
aggregate_all	Todos verdaderos	int[]	int
aggregate_any	Alguno verdadero	int[]	int
aggregate_none	Ninguno verdadero	int[]	int

3.4 Funciones Enteras

Función	Descripción	Entrada	Salida
aggregate_sum_int	Suma entera	int[]	int
aggregate_min_int	Mínimo entero	int[]	int
aggregate_max_int	Máximo entero	int[]	int
aggregate_avg_int	Promedio de enteros	int[]	double

4. Proceso de Compilación

4.1 Modificación del Makefile

Se agregó el módulo al sistema de compilación:

```
# Regla principal - incluye aggregate_functions.o
bridge: bridge_gecode.tab.c lex.yy.c json_reader.o expr_parser.o \
    expr_eval.o json_output.o cJSON.o aggregate_functions.o
    $(CC) $(CFLAGS) -o bridge bridge_gecode.tab.c lex.yy.c \
        json_reader.o expr_parser.o expr_eval.o json_output.o \
        cJSON.o aggregate_functions.o -lm

# Regla de compilación del módulo
aggregate_functions.o: aggregate_functions.c aggregate_functions.h
    $(CC) $(CFLAGS) -c aggregate_functions.c
```

4.2 Pasos de Compilación

Paso 1: Limpiar archivos previos

```
make clean
```

Paso 2: Compilar módulo de funciones agregadas

```
gcc -Wall -std=c11 -g -lm -c aggregate_functions.c
```

Salida: - aggregate_functions.o (código objeto, 14 KB)

Paso 3: Compilar todo el proyecto

```
make
```

Proceso: 1. Genera parser con Bison 2. Genera lexer con Flex 3. Compila módulos a .o: - json_reader.o - expr_parser.o - expr_eval.o - json_output.o - cJSON.o - **aggregate_functions.o** < NUEVO 4. Linkea todo en ejecutable bridge

4.3 Verificación

```
$ ls -lh aggregate_functions.*
-rw-rw-r-- 1 user user 5.3K aggregate_functions.c
-rw-rw-r-- 1 user user 2.7K aggregate_functions.h
-rw-rw-r-- 1 user user 14K aggregate_functions.o    < Código objeto
```

5. Uso de Funciones Agregadas

5.1 Desde Código C

```
#include "aggregate_functions.h"

void ejemplo() {
    double datos[] = {1.5, 2.3, 4.7, 3.1, 5.9};
    size_t n = 5;

    double suma = aggregate_sum(datos, n);      // 17.5
    double promedio = aggregate_avg(datos, n);  // 3.5
    double minimo = aggregate_min(datos, n);    // 1.5
    double maximo = aggregate_max(datos, n);    // 5.9
    double desv = aggregate_stdev(datos, n);    // 1.67
}
```

5.2 Desde Expresiones JSON (Futuro)

Cuando se integren al parser, se podrán usar así:

```
{
  "precision": 2,
  "variables": [
    {"nombre": "valores", "tipo": "array", "value": [1.5, 2.3, 4.7, 3.1, 5.9]}
  ],
  "expresiones": [
    "SUM(valores)",
    "AVG(valores)",
    "MIN(valores) + MAX(valores)",
    "STDEV(valores) < 2.0"
  ]
}
```

6. Detalles de Implementación

6.1 SUM - Suma de Elementos

```
double aggregate_sum(const double *array, size_t length) {
    if (!array || length == 0) return 0.0;

    double sum = 0.0;
    for (size_t i = 0; i < length; i++) {
        sum += array[i];
    }
    return sum;
}
```

```
}
```

Características: - Validación de entrada (!array) - Manejo de array vacío - Acumulador inicializado - Complejidad: O(n)

6.2 AVG - Promedio

```
double aggregate_avg(const double *array, size_t length) {  
    if (!array || length == 0) return 0.0;  
    return aggregate_sum(array, length) / (double)length;  
}
```

Características: - Reutiliza aggregate_sum - Conversión explícita a double - División segura

6.3 MEDIAN - Mediana

```
double aggregate_median(double *array, size_t length) {  
    if (!array || length == 0) return 0.0;  
  
    /* Ordenar el array */  
    qsort(array, length, sizeof(double), compare_doubles);  
  
    /* Calcular mediana */  
    if (length % 2 == 0) {  
        size_t mid = length / 2;  
        return (array[mid - 1] + array[mid]) / 2.0;  
    } else {  
        return array[length / 2];  
    }  
}
```

Características: - Modifica el array (ordenamiento) - Usa qsort de stdlib - Maneja longitud par e impar - Complejidad: O(n log n)

6.4 VARIANCE - Varianza

```
double aggregate_variance(const double *array, size_t length) {  
    if (!array || length == 0) return 0.0;  
  
    double mean = aggregate_avg(array, length);  
    double sum_sq_diff = 0.0;  
  
    for (size_t i = 0; i < length; i++) {  
        double diff = array[i] - mean;  
        sum_sq_diff += diff * diff;  
    }  
}
```

```

    return sum_sq_diff / (double)length;
}

```

Fórmula: $\sigma^2 = \text{SUM}(\xi - \mu)^2 / n$

6.5 ALL/ANY/NONE - Operadores Lógicos

```

int aggregate_all(const int *array, size_t length) {
    if (!array || length == 0) return 0;

    for (size_t i = 0; i < length; i++) {
        if (!array[i]) {
            return 0; /* Short-circuit */
        }
    }
    return 1;
}

```

Características: - Short-circuit evaluation - Retorno temprano - Eficiente:
 $O(n)$ en peor caso, $O(1)$ en mejor caso

7. Ventajas de la Arquitectura Modular

7.1 Separación de Responsabilidades

Módulo	Responsabilidad
bridge_gecode.y/l	Parsing y sintaxis
expr_eval.c	Evaluación de expresiones
aggregate_functions.c	Operaciones agregadas

7.2 Beneficios

1. Mantenibilidad

- Cambios aislados
- Fácil localización de bugs
- Código organizado

2. Testabilidad

- Cada módulo se puede testear independientemente
- Unit tests más simples

3. Reutilización

- Las funciones pueden usarse en otros proyectos
- API clara y documentada

4. Extensibilidad

- Agregar nuevas funciones es trivial
- Sin modificar código existente

5. Compilación Eficiente

- Solo recompila lo que cambia
 - Código objeto reutilizable
-

8. Integración con el Parser (Próximos Pasos)

8.1 Modificar bridge_gecode.y

Agregar tokens para funciones agregadas:

```
%token SUM AVG MIN MAX MEDIAN VARIANCE STDEV
%token ALL ANY NONE COUNT
```

expresion:

```
| SUM '(' expresion ')' { $$ = crear_nodo_agregado(TIPO_SUM, $3); }
| AVG '(' expresion ')' { $$ = crear_nodo_agregado(TIPO_AVG, $3); }
| MIN '(' expresion ')' { $$ = crear_nodo_agregado(TIPO_MIN, $3); }
/* ... más reglas */
```

8.2 Modificar bridge_gecode.l

Agregar reconocimiento de palabras clave:

```
"SUM"      { return SUM; }
"AVG"      { return AVG; }
"MIN"      { return MIN; }
"MAX"      { return MAX; }
/* ... más tokens */
```

8.3 Extender expr_eval.c

Agregar evaluación de funciones agregadas:

```
#include "aggregate_functions.h"

ResultadoEval* evaluar_expresion(Nodo *expr) {
    switch (expr->tipo) {
        case TIPO_SUM:
            /* Evaluar argumento, obtener array, llamar aggregate_sum */
            break;
        /* ... más casos */
    }
}
```

9. Ejemplos de Uso

9.1 Estadísticas Descriptivas

```
{
  "precision": 2,
  "variables": [
    {"nombre": "notas", "tipo": "array_float",
     "value": [7.5, 8.2, 6.9, 9.1, 7.8, 8.5]}
  ],
  "expresiones": [
    "AVG(notas)",           // Promedio: 8.0
    "MIN(notas)",           // Mínimo: 6.9
    "MAX(notas)",           // Máximo: 9.1
    "MEDIAN(notas)",        // Mediana: 8.0
    "STDEV(notas)"          // Desv. Est.: 0.71
  ]
}
```

9.2 Análisis de Conjuntos

```
{
  "variables": [
    {"nombre": "equipo", "tipo": "set",
     "value": ["alice", "bob", "charlie"]}
  ],
  "expresiones": [
    "COUNT(equipo)",       // 3
    "COUNT(equipo) >= 2"   // true
  ]
}
```

9.3 Validación Lógica

```
{
  "variables": [
    {"nombre": "checks", "tipo": "array_bool",
     "value": [true, true, false, true]}
  ],
  "expresiones": [
    "ALL(checks)",          // false
    "ANY(checks)",          // true
    "NONE(checks)"          // false
  ]
}
```

10. Testing y Validación

10.1 Test Unitario Básico

```
#include "aggregate_functions.h"
#include <assert.h>
#include <stdio.h>

void test_sum() {
    double arr[] = {1.0, 2.0, 3.0, 4.0, 5.0};
    double resultado = aggregate_sum(arr, 5);
    assert(resultado == 15.0);
    printf("test_sum: PASSED\n");
}

void test_avg() {
    double arr[] = {2.0, 4.0, 6.0, 8.0};
    double resultado = aggregate_avg(arr, 4);
    assert(resultado == 5.0);
    printf("test_avg: PASSED\n");
}

int main() {
    test_sum();
    test_avg();
    printf("All tests passed!\n");
    return 0;
}
```

10.2 Compilar y Ejecutar Tests

```
gcc -o test_aggregate test_aggregate.c aggregate_functions.o -lm
./test_aggregate
```

11. Documentación de API

11.1 Convenciones

- **Prefijo:** Todas las funciones usan `aggregate_`
- **Parámetros:** Arrays como `const` cuando no se modifican
- **Retorno:** `double` para funciones numéricas, `int` para lógicas
- **Manejo de errores:** Retorna 0.0 o 0 para entradas inválidas

11.2 Complejidad Computacional

Función	Complejidad	Notas
SUM	$O(n)$	Un solo paso
AVG	$O(n)$	Usa SUM
MIN/MAX	$O(n)$	Búsqueda lineal
MEDIAN	$O(n \log n)$	Requiere ordenamiento
VARIANCE	$O(n)$	Dos pasos: media y varianza
STDEV	$O(n)$	Usa VARIANCE + sqrt
ALL/ANY/NONE	$O(n)$	Con short-circuit

12. Conclusiones

12.1 Logros

- OK Módulo independiente de funciones agregadas
- OK Código objeto generado (`aggregate_functions.o`)
- OK 15+ funciones implementadas
- OK Integración con Makefile
- OK Compilación exitosa
- OK API clara y documentada

12.2 Próximos Pasos

1. Integrar con parser (yacc/lex)
2. Extender evaluador de expresiones
3. Crear tests JSON completos
4. Agregar más funciones:
 - PERCENTILE(array, p)
 - MODE(array)
 - RANGE(array)
 - CORRELATION(array1, array2)

12.3 Recursos

- **Código fuente:** `aggregate_functions.c/h`
 - **Código objeto:** `aggregate_functions.o` (14 KB)
 - **Makefile:** Actualizado con reglas de compilación
 - **Documentación:** Este manual
-

Fin del Manual Técnico